

WorldGen: an agent that builds simulated worlds

The problem

We test AI agents against **worlds**: stateful replicas of real software (a helpdesk, a CRM, a payments API). An agent calls the world's API, the world's state changes, and a grader reads the end state to score the run.

Building a good world by hand takes days. Build two things that make it take minutes:

1. A lightweight world engine

Runs and grades worlds.

2. WorldGen

An agent that turns a description of some software into a world for that engine.

BEFORE

User hand-builds data model, API, logic, data, tasks



Days of work, mostly abandoned

AFTER

"A helpdesk with SLA tiers and on-call escalation"



WorldGen

↓ world ↑ errors

World engine



A working, graded world

You get model credits and a Slack channel with our team. Language, libraries and design are your choice.

The world engine

Keep it small and strict. You design the format.

A world contains

- A **data model**: entities, fields, types, keys and relationships.
- An **API**: routes mapped to standard operations (list, get, create, update, delete) or to custom logic.
- **Custom logic** for workflows that go beyond plain reads and writes (e.g. escalating a ticket).
- **Seed data**: the world's starting state.
- **Tasks**: an instruction for an agent, plus a grader that scores the end state from 0 to 1.

Minimum features

- **Check**: validate a world before running it, with errors precise enough for a model to fix them.
- **Serve**: answer the world's API over HTTP from a fresh copy of the seed data.
- **Enforce**: refuse any write that breaks the data model. A failed call leaves no partial changes.
- **Deterministic**: the same world and task always start from the same state. Time is controlled by the engine, not the wall clock.
- **Inspect and reset**: dump the current state, reset to the seed, and see a log of the calls made.
- **Grade**: score a task from the end state, and confirm the grader gives full marks to a reference solution and zero to doing nothing.

WorldGen

WorldGen is an agent that turns a request into a working world. It should feel like handing the job to a careful engineer: it asks the right questions of the input, builds in stages, checks its own work, and tells you what it did.

Inputs

Any one of:

- **A description:** “an IT asset tracker with laptops, assignments, repair tickets and a quarterly audit”. WorldGen fills the gaps with sensible choices and writes them down.
- **An API spec** (e.g. OpenAPI): the world should follow the spec's paths, shapes and errors, optionally narrowed to part of it.
- **Sample data** (e.g. a CSV export): WorldGen infers the data model from it and uses it to shape the seed data.

Outputs

A world for your engine containing:

- A data model and API that resemble the real software.
- Custom logic for at least one real workflow (something with states and rules, not just storage).
- Realistic, internally consistent seed data: plausible values, relationships that resolve, a believable mix of states, enough rows that searching and paging matter.
- At least three graded tasks at different difficulty levels, each with a reference solution.
- A short report: what was built, what was assumed and why, what was left out, and proof the checks and graders pass.

How it works

Build in stages, checking after each one, rather than generating everything in a single prompt:

- 1 **Understand** the input and draft a plan: entities, routes, workflows, and the tasks a user of this software would care about.
- 2 **Model** the data and API, then check.
- 3 **Implement** the workflow logic, then check and test.
- 4 **Generate** seed data, then check.
- 5 **Write tasks**, graders and reference solutions, then confirm each grader gives full marks to its solution and zero to doing nothing.
- 6 **Report**.

Minimum features

- **One command** from input to finished world.

- **Plan first:** the plan is saved so a human can read it, and later stages follow it.
 - **Self-repair loop:** when the engine reports a failure, WorldGen feeds it back to the model and fixes it, retrying within a budget.
 - **Knows when to stop:** if it can't produce a passing world, it says why instead of handing over a broken one.
 - **Uses the engine as its judge:** it never edits the engine and never grades its own work by asking the model.
 - **Iterates:** given an existing world and a change request ("add refunds"), it updates the world rather than starting over.
 - **Observable:** logs each stage, every repair attempt, time taken and model cost.
 - **Configurable:** model and budget are settings, not hard-coded.
-

What makes a good world

- **Reads and writes.** Real workflows change state; a read-only world is a lookup table.
 - **Faithful to the real software,** not to what is convenient for the agent calling it.
 - **No silent guessing.** Assumptions are written down.
 - **Graders that discriminate.** A correct solution scores full marks; doing nothing scores zero.
-

Deliverables

- A repository with the engine and WorldGen, each runnable with one command.
- A short design doc explaining the world format, the engine's guarantees, and WorldGen's loop.
- One world built by hand, and the worlds WorldGen generated from the prompts we send you.

We'll run WorldGen live on prompts you haven't seen at the end of the trial.

Questions are welcome. Where the spec is ambiguous, make a call, write it down, and keep going.